# HomeWorldz Roadmap

This roadmap describes the major implementation sequence for HomeWorldz. It is organized at three levels: phases, milestones within each phase, and major work items within each milestone. `PLAN.md` remains the detailed engineering checklist, while `FEATURES.md` records intentional product differences and the ADRs record architectural decisions.

Checkboxes describe the present state, not a promise of a release date. A milestone is complete only when its automated tests and applicable Firestorm acceptance tests pass.

## Phase 1: Playable Single Region

### Platform foundation

- ✓ Establish the C++20 region server, Go grid service, PostgreSQL central state, and SQLite plus filesystem region-local state.
- ✓ Define configuration, bootstrap, health, authentication, logging, CI, and service contracts.
- ✓ Implement region registration, leases, discovery, login sessions, and online presence.
- ✓ Establish the authoritative fixed-step scene loop and durable scene snapshots.

### Viewer connection and appearance

- ✓ Complete the minimum supported Firestorm login, UDP circuit, handshake, capabilities, event delivery, terrain, chat, and static-object flow.
- ✓ Provide default body parts, clothing, Current Outfit links, legacy avatar baking, and persistent appearance across relogs.
- ✓ Provide a read-only system Library with default avatar and terrain content.
- ☐ Keep avatar appearance stable while moving, flying, sitting, teleporting, and crossing regions.

### Authoritative avatar movement

- ✓ Decode movement, camera, jump, and flight controls and persist provisional avatar state.
- ✓ Begin streaming authoritative position, velocity, and rotation changes back to viewers.
- ☐ Complete viewer-visible walking, turning, jumping, stopping, flight toggling, ascent, and descent without requiring a relog.

- [ ] Add animation-state selection and synchronization for standing, walking, running, jumping, falling, flying, hovering, and landing.
- [ ] Reconcile viewer prediction with the authoritative region position without visible snapping or drift.

### Basic avatar physics

- [ ] Integrate a Jolt avatar capsule into the production scene loop.
- [ ] Sample terrain continuously for grounding rather than retaining only the height at the login position.
- [ ] Support terrain walking, slopes, steps, falling, jumping, landing, flight, and collision-safe motion.
- [ ] Collide avatars with static and dynamic scene objects while preserving practical viewer movement behavior.
- [ ] Persist and restore position, orientation, velocity, flight state, and grounded state safely.

### Core inventory, assets, and objects

- [x] Implement AIS v3 inventory fetch and mutation, Library outfit copying, Current Outfit links, folder operations, and Trash lifecycle operations.
- [x] Implement free texture upload, required creator provenance, content-addressed assets, origin registration, and region replication.
- [x] Implement primitive rez, edit, permissions, ownership, take, delete, restore, and restart persistence.
- [ ] Complete linksets, child-prim transforms, object inventory, duplication, return, and derez edge cases.
- [ ] Add the remaining fundamental content types needed by appearance, building, attachments, and scripts, including animations, sounds, gestures, notecards, landmarks, and LSL source.

## Phase 2: Interactive Physical World

### Production physics integration

- [ ] Make Jolt the default production physics world while retaining the engine-independent plugin boundary.
- [ ] Create, update, sleep, wake, remove, and restore physical bodies from authoritative scene changes.
- [ ] Synchronize physical transforms and velocities to viewers at suitable rates with interest-aware throttling.
- [ ] Implement collision filtering, material behavior, phantom and temporary objects, volume detection, and collision events.

- [ ] Verify deterministic-enough restart and handoff behavior through shared physics acceptance scenarios.

### Attachments and sitting

- [ ] Attach inventory objects to named avatar attachment points with stable local transforms, permissions, ownership, and persistence.
- [ ] Represent worn attachments as part of the authoritative avatar bundle and restore them on login.
- [ ] Implement sit targets, avatar seating, unsit, camera placement, and seated animation state.
- [ ] Support avatars as seated attachments to object linksets so their world transforms follow the root object correctly.
- [ ] Define lifecycle ordering for attachment, seated-avatar, physics, viewer, and later script events.

### Vehicles and physical objects

- [ ] Implement stable dynamic-object movement, editing, taking, and restoration without losing physics state.
- [ ] Add the Second Life vehicle parameter model required by LSL vehicles.
- [ ] Synchronize driver controls, vehicle motion, cameras, passengers, and seated-avatar transforms.
- [ ] Preserve object, linkset, inventory, permission, passenger, and physical state as one transferable vehicle bundle.
- [ ] Add load, tunneling, stacking, recovery, and abusive-object safeguards.

### Parcels and local authority

- [ ] Implement parcel geometry, ownership, access, landing points, media, environment, and object accounting.
- [ ] Enforce build, rez, entry, script, damage, push, and object-return policy at authoritative boundaries.
- [ ] Implement estate and region settings needed for terrain, access, maturity, restart, and emergency administration.
- [ ] Apply permissions recursively and consistently to linksets, object contents, attachments, and inventory transfers.

## Phase 3: Connected Multi-Region World

### Region topology and variable size

- [ ] Represent neighboring regions, coordinates, extents, public endpoints, maturity, and online state in grid discovery.

- [ ] Support exactly 1x1 (256 m), 2x2 (512 m), and 4x4 (1024 m) regions.
- [ ] Generalize terrain, physics bounds, viewer coordinates, storage, map tiles, and interest management to the three supported sizes.
- [ ] Prevent overlaps and invalid neighbor layouts and define behavior beside offline or differently sized regions.

### Teleports and avatar crossings

- [ ] Build an authenticated, idempotent two-region handoff transaction with a transit UUID, generation, prepare, accept, activate, and rollback stages.
- [ ] Implement local and remote teleports, destination validation, viewer circuit establishment, failure recovery, and arrival placement.
- [ ] Cross a walking or flying avatar between adjacent regions while preserving appearance, controls, velocity, camera, and session continuity.
- [ ] Transfer the complete attachment set with the avatar and prevent duplicate activation at source and destination.
- [ ] Handle disconnects, destination failure, retries, stale transit records, and reconciliation after process restart.

### Object and vehicle crossings

- [ ] Cross individual objects and complete linksets without changing creator, owner, permissions, inventory, or physical state.
- [ ] Cross scripted and unscripted attachments as part of their avatar bundle.
- [ ] Cross vehicles while preserving linear and angular motion, vehicle parameters, and object inventory.
- [ ] Transfer a vehicle and all seated avatars as one coordinated bundle, with no passenger briefly becoming authoritative in both regions or neither.
- [ ] Establish event and collision cutoffs so crossing does not duplicate or silently lose observable actions.

### World navigation

- [ ] Generate and serve region and world map tiles for all supported region sizes.
- [ ] Implement map-block discovery, landmark resolution, home location, and teleport routing.
- [ ] Add region and parcel search sufficient to find and reach destinations.
- [ ] Show friends and authorized users useful presence and location without leaking restricted information.

## Phase 4: LSL Scripting

## Language and compiler

- [ ] Inventory the complete Second Life LSL language and built-in surface plus Halcyon/InWorldz extensions, explicitly excluding OpenSimulator extensions.
- [ ] Implement the handwritten lexer, parser, semantic analysis, diagnostics, and versioned HomeWorldz bytecode compiler.
- [ ] Store LSL source with creator provenance and cache immutable bytecode by source hash, compiler version, and runtime ABI.
- [ ] Build compatibility tests for syntax, types, conversions, lists, strings, states, constants, built-ins, and observable errors.

## Cooperative runtime and resource control

- [ ] Implement the single-threaded C++ bytecode VM on the authoritative region thread with explicit instruction-level execution state.
- [ ] Schedule scripts fairly using bounded weighted instruction and wall-clock budgets with no native thread per script.
- [ ] Enforce memory, stack, call-depth, event-queue, string, list, payload, owner, object, and parcel limits.
- [ ] Make slow host operations asynchronous and represent waits as serializable tokens or continuations.
- [ ] Add operator metrics, throttling, diagnostics, stopping, resetting, and isolation for inefficient or faulty scripts.

## Events and region integration

- [ ] Implement object lifecycle, touch, timer, listen, sensor, control, permission, inventory, changed, link-message, collision, land-collision, attachment, and moving events.
- [ ] Implement bounded LSL host functions for scene, physics, inventory, communication, parcel, avatar, HTTP, and data operations.
- [ ] Preserve Second Life event ordering and delay semantics where observable and document intentional HomeWorldz differences.
- [ ] Integrate script ownership and permissions with linksets, attachments, seated avatars, parcels, and estate policy.

## Script persistence and crossings

- [ ] Serialize bytecode identity, instruction pointer, stacks, frames, globals, current event, event queue, timers, listens, permissions, and pending work in a compact versioned binary format.
- [ ] Stop and restore a script after any completed bytecode instruction without relying on the native C++ stack.

- [ ] Snapshot scripts atomically with their attachment, object, or vehicle physics bundle.
- [ ] Cross heavily scripted attachments and vehicles within defined latency, memory, duplication, and event-loss limits.
- [ ] Version the runtime ABI and provide safe upgrade, incompatibility, and rollback behavior for stored script state.

# Phase 5: Social and Creator Platform

## Identity, profiles, and communication

- [ ] Implement user-visible names, profiles, interests, images, privacy, and account administration.
- [ ] Implement direct messages, offline messages, group chat, conference chat, mute/block behavior, and delivery history where appropriate.
- [ ] Implement friendship, calling cards, presence permissions, and offers.
- [ ] Add abuse reporting and the minimum moderation evidence needed by grid operators.

## Groups, roles, and shared ownership

- [ ] Implement groups, roles, powers, membership, invitations, notices, and group communication.
- [ ] Support group-owned land and objects without weakening creator provenance or transfer permissions.
- [ ] Apply group powers consistently to parcels, estates, object editing, inventory sharing, and moderation.
- [ ] Audit sensitive group and ownership changes.

## Content creation and inventory breadth

- [ ] Complete viewer building workflows for linksets, materials, mesh, sculpt, animation, sound, gesture, notecard, landmark, and script content.
- [ ] Implement uploads, validation, dependencies, creator attribution, asset replication, and inventory creation for each supported asset type.
- [ ] Add outfit creation and editing beyond the initial default-avatar flow.
- [ ] Provide bulk inventory, search, copy, transfer, export-policy, recovery, and large-inventory performance behavior.

## Economy and marketplace boundary

- [ ] Define whether credits remain display-only or become a transferable grid balance before implementing paid behavior.
- [ ] If enabled, implement auditable balances, idempotent transactions, object

sales, parcel payments, gifts, refunds, and operator controls.

- [ ] Keep texture uploads free and preserve a useful no-economy deployment mode.
- [ ] Treat external payment processing and marketplace integration as separate, explicitly approved security projects.

# Phase 6: Reliable Operations and Distribution

## Grid and region packages

- [ ] Produce separate versioned grid-owner and region-owner packages containing prebuilt executables, runtime dependencies, examples, bootstrap tools, and end-user installation guides.
- [ ] Support clean install, unattended install, upgrade, downgrade where safe, uninstall, and configuration preservation.
- [ ] Sign release artifacts, publish checksums and provenance, and generate a machine-readable release manifest.
- [ ] Validate supported Windows and Linux installations without requiring a source checkout or development toolchain.

## Backups, upgrades, and reconciliation

- [ ] Back up and restore PostgreSQL grid state, region SQLite state, assets, terrain, configuration, and compatible runtime state.
- [ ] Test full-grid, single-region, and selected-user recovery with documented recovery-point and recovery-time expectations.
- [ ] Version schemas and protocols and support rolling grid and region upgrades within a documented compatibility window.
- [ ] Reconcile leases, presence, inventory, assets, crossings, and duplicated or orphaned state after crashes or partial restores.

## Observability and administration

- [ ] Provide metrics, structured logs, traces, health detail, dashboards, and actionable alerts for grid and region owners.
- [ ] Add command-line and authenticated web administration for users, regions, estates, assets, inventory repair, scripts, crossings, and moderation.
- [ ] Record tamper-evident audit events for privileged and security-sensitive operations.
- [ ] Define capacity indicators and load-shedding behavior before a region becomes unresponsive.

## Security and deployment hardening

- [ ] Add transport encryption, service identity, credential rotation, scoped authorization, secret-management guidance, and secure defaults for non-local deployments.
- [ ] Validate all viewer, inter-region, asset, inventory, script, and operator inputs against resource-exhaustion and malformed-data attacks.
- [ ] Add dependency, artifact, and configuration scanning plus a vulnerability response and supported-version policy.
- [ ] Perform fault-injection, abuse, denial-of-service, and recovery testing before describing a release as production-ready.

# Phase 7: Scale, Compatibility, and Ecosystem

## Performance and scale

- [ ] Establish repeatable concurrency, scene-complexity, physics, inventory, asset, crossing, script, and network benchmarks.
- [ ] Implement interest management, packet prioritization, backpressure, and adaptive update rates for crowded or complex regions.
- [ ] Scale central services horizontally where measurements justify it while keeping each region's authority unambiguous.
- [ ] Publish tested capacity envelopes rather than relying on nominal limits.

## Compatibility

- [ ] Maintain conformance tests against the pinned supported Firestorm release and evaluate newer releases deliberately.
- [ ] Add read-only legacy inventory access only if its older-viewer benefit justifies the maintenance cost; AIS v3 remains authoritative.
- [ ] Validate Halcyon/InWorldz LSL extensions without admitting OpenSimulator scripting extensions accidentally.
- [ ] Document import and migration tools separately from live legacy service or database compatibility.

## Physics and service extensions

- [ ] Promote the existing PhysX 5 adapter to an optional supported physics plugin after it passes the same production scenarios as Jolt.
- [ ] Stabilize versioned plugin contracts only for boundaries with demonstrated operational value.
- [ ] Define safe extension points for grid services without exposing region authority or script execution to untrusted in-process plugins.
- [ ] Maintain deterministic transfer and persistence contracts across every supported physics implementation.

## Release readiness

- [ ] Publish administrator, region-owner, creator, scripter, and contributor documentation appropriate to the supported feature set.
- [ ] Run sustained multi-region worlds with real viewers, scripts, attachments, vehicles, failures, upgrades, and restores.
- [ ] Resolve all release-blocking correctness, data-loss, permissions, crossing, security, and viewer-compatibility findings.
- [ ] Define the supported platform matrix, compatibility guarantees, upgrade policy, and long-term maintenance expectations for the first stable release.